

IO er et spesialtilfelle av side-effekter “ute i verden”

- ▶ $f :: \text{Tilstand TypeA} \rightarrow \text{TilstandIO TypeB}$
- ▶ IO markerer at $f(a)$ har en side-effekt, mens TypeB er det f “egentlig” returnerer. Ofte, betyr det ingenting og da er $\text{TypeB} = \text{tomt produkt} = ()$, mens funksjonen returner en null tuppel $() :: ()$, dvs. $f :: \text{TypeA} \rightarrow \text{IO } ()$.
- ▶ `import System.IO` øverst i programfilen.
- ▶ “Funksjoner” med IO i returtype heter “aksjoner”, f.eks.:

<code>putChar</code>	<code>::</code>	<code>Char</code>	<code>-></code>	<code>IO ()</code>
<code>putStr</code>	<code>::</code>	<code>String</code>	<code>-></code>	<code>IO ()</code>
<code>putStrLn</code>	<code>::</code>	<code>String</code>	<code>-></code>	<code>IO ()</code>
<code>print</code>	<code>::</code>	<code>Show a => a</code>	<code>-></code>	<code>IO ()</code>
<code>getChar</code>	<code>::</code>			<code>IO Char</code>
<code>getLine</code>	<code>::</code>			<code>IO String</code>
- !! Generelt, gjelder ikke: `getChar == getChar`
- ▶ `putStr (“Velkommen “ ++ getLine)` er ulovlig – fordi feiltypet Aksjoner kan ikke blandes med/kalles av (vanlige) funksjoner.



Aksjoner utføres

- ▶ hvis programfilen `fil.hs` har en aksjon `main = ...`:
 - den kan kompileres: `ghc fil.hs`
 - og deretter kan binærfil eksekveres fra terminalvindu:
`> ./fil` (avhengig av systemet; `fil.exe`)
- ▶ når man skriver aksjon(ens navn) i `ghci`

`fil.hs`: `main``navn` = `putStrLn "Hello"`

`> ghci fil`

`> main``navn`

`Hello``>`

`>`

`fil2.hs` `navn2` = `"Hei"`

`> ghci fil2`

`> navn2`

`"Hei"`

`>`

IO: sekvensiering av aksjoner

► `putStr "Hei. " >> putStr "Går det bra" >> putStrLn "?"`

$\simeq$ `putStrLn "Hei. Går det bra?"`

```
do putStr "Hei."  
   putStr "Går det bra"  
   putStrLn "?"
```

► første ikke-blank etter `do` bestemmer indentering

```
hei = do  
    putStr "Hei."  
    putStr "Går det bra"  
    putStrLn "?"
```

`do` lager en **sekvens** av aksjoner – som ingenting annet i FP

IO: noen aksjoner gir et resultat (input)

- ▶ `<-` lager en aksjon som
 - fjerner IO og
 - trekker ut aksjonens “egentlige resultatet”
- ▶ `aksjon :: IO TypeB`
`res <- aksjon`
`res :: TypeB`
- ▶ `res <- getLine`
- `main = do`
`res1 <- putStrLn "Hva heter du?" >>`
`navn <- getLine >>` feil
`putStrLn ("Velkommen " ++ navn)` :: IO ()
- `do` samler en sekvens av aksjoner (ikke funksjoner!) i én aksjon som returnerer det samme, som den siste aksjonen i listen
- ▶ aksjonsekvenser settes typisk i én unik `main` (hovedblokk, som eksekveres ved å kalle `main`, evt. i det programmet lastes)

Aksjoner kan fritt kalle funksjoner

- les linje fra skjermen og vis alle ord reversert
(inntil innlest linje er tom)

```
main = do
```

```
  line <- getLine
```

```
  if not null line then do
```

```
    putStrLn (reverseWords line)
```

```
    main
```

```
  else ???()return ()
```

► reverseWords :: String -> String

```
reverseWords = unwords . map reverse . words
```

► words :: String -> [String] – del strengen ved blanker

unwords :: [String] -> String – én streng med blanker

► main :: IO ()

► return x – lager en aksjon ut av verdien x; return :: t -> IO t

Kontrollstrukturer for aksjoner, og return

- ▶ aksjoner kan kombineres i vanlige kontrolstrukturer: if, case, rekursjon, osv.

`getLine` :: IO String

```
getLine = do x <- getChar
            if x == '\n' then return []
            else
                do xs <- getLine
                   return (x:xs)
                   return 5
                   return (x:xs)
```

– dette forandrer ingenting

`return` z – lager en aksjon ut av verdien z; `return` :: t -> IO t

- ▶ Den hverken gjør eller returnerer noenting!

Mer aksjonsyntaks, fra funksjoner

```
import Data.Char

main = do
    putStrLn "Ditt fornavn?"
    fornavn <- getLine
    putStrLn "Ditt etternavn?"
    etternavn <- getLine
    let stortFor = map toUpper fornavn
        stortEtt = map toUpper etternavn           – uten 'in'
    putStrLn ($) "Hei, " ++ stortFor ++ stortEtt ++ ". Alt ok?"

▶ a <- aksjon                                     – gir en ny aksjon
  – aksjonstype = IO (type til a)

▶ let a = verdi (funksjon arg) (uten 'in')        – gir en ny aksjon
  – type til verdi = type til a

▶ let a = putStr "Hei. " in a >> a >> a
  – gir tilfeldigvis en aksjon (grunnet a og >> etter 'in')
```

Et par eksempler

1. fra boken: 10.4

> adder

Hvor mange tall? 3

2

5

7

Summen er: 14

Endre programmet fra punkt 1 slik at

2. man ikke oppgir antall tall i første linje, men bare tallene, som avsluttes med 0

– da skal programmet vise summen av alle oppgitte tall.

3. Alle tall skrives i en linje – i det man trykker CR (Enter), skal programmet vise summen av tallene.

Aksjoner

- ▶ har sideeffekter, representert ved **IO** i returtype
- ▶

putChar	::	Char	->	IO ()
putStr	::	String	->	IO ()
putStrLn	::	String	->	IO ()
print	::	Show a => a	->	IO ()
getChar	::			IO Char
getLine	::			IO String
- ▶ Vi kan lage aksjoner selv:
 - var **<-** aksjon
 - let** var = funksjonskall
 - return** verdi
- ▶ En sekvens av aksjoner samles i én aksjon med **do**
 - denne aksjonen returnerer det som siste aksjonen i sekvensen

Lister av aksjoner

- ▶ act Aksjoner er “ting” som andre verdier eller funksjoner – elementer av typer

```
[print 1, print 2, print 3] :: [IO ()]
```

? Skriv alle tall fra 1 til n

▶ vis n = [print i | i <- [1..n]]

> vis 2

No instance for (Show (IO ())) arising from a use of 'print'

```
[print i | i <- [1..n] ] :: [IO ()]
```

▶ visS n = **sequence** [print i | i <- [1..n]]

sequence :: Monad m => [mIO a] -> mIO [a]

> visS 2

1

2

[(),()]

Lister av aksjoner

- ▶ `visS n = sequence [print i | i <- [1..n] ]`
- ▶ `visS 2`
1
2
[(),()]
- ▶ `sequence :: [IO a] -> IO [a]`
- ▶ `sequence_ :: Monad m => [mIO a] -> mIO ()`
- ▶ `visA n = sequence_ [print i | i <- [1..n] ]`
- ▶ `visA 2`
1
2
- ▶ `delay n = sequence_ [return () | i <- [1..n] ]`

Lister av aksjoner og aksjoner på lister

- ▶ `visS n = sequence [print i | i <- [1..n] ]`

`visSBCD 2`

`1`

`2`

`[(),()]`

`sequence :: [IO a] -> IO [a]`

`print :: Show a => a -> IO ()`

- ▶ `visB n = sequence_ $ map print [1..n]` `:: [IO ()]:: IO [()]`

`map :: (a -> b) -> [a] -> [b]`

- ▶ `visC n = mapM print [1..n]`

`mapM :: (a -> IO b) -> [a] -> IO [b]`

- ▶ `visD n = mapM_ print [1..n]`

`mapM_ :: (a -> IO b) -> [a] -> IO ()`

`mapM_ :: Monad m => (a -> m b) -> [a] -> m ()`

Filer

```
main = do
    putStrLn "Filnavn?"
    filnavn <- getLine

    1. filen <- openFile filnavn ReadMode
    der openFile :: FilePath -> Mode -> IO Handle, der
    type FilePath = String
    data Mode = ReadMode | WriteMode | AppendMode |
                                                (ReadWriteMode)
    2. innhold <- hGetContents filen
                                   - hGetContents:: Handle -> IO String
    ...

    3. hClose filen
eller: .....
    innhold <- readFile filnavn
    ...

    der readFile :: FilePath -> IO String
```

Skriving til filer

- ▶ `filen <- openFile filNavn`
`WriteMode / AppendMode / (ReadWriteMode)`
`/ (ReadMode)`

gir en “handle” til filen

som man kan skrive til med

`hPutStr, hPutStrLn :: Handle -> String -> IO ()`

og som må lukkes til slutt: `hClose` filen (dvs. `Handle`)

```
do fil <- openFile navn ReadMode
   tekst <- hGetContents fil
   hClose fil                                     = tekst <- readFile navn
```

```
do fil <- openFile navn WriteModeAppendMode
   hPutStr fil string
   hClose fil                                     = writeFileappendFile fil string
```

- ▶ `writeFile, appendFile :: FilePath -> String -> IO ()`
`writeFile` filNavn tekst – erstatter alt i filen med tekst

Modifisering av en fil

```
flyttLn fn = do                                --flytt første linjen til slutt
  innhold <- readFilereadWriteFile fn
  let (l1:rest) = lines innhold
      nytt = unlines (rest++[l1])
  writeFile fn nytt ..... Exception (resource busy...)
```

```
flyttLn2 fn = do
  fh <- openFile fn ReadMode
  innhold <- hGetContents fh
  let (l1:rest) = lines innhold
      nytt = unlines (rest++[l1])
  hClose fh
  fh2 <- openFile fn WriteMode
  hPutStr fh2 nytt ..... Exception (resource busy...)
  hClose fh2
```

Aksjoner `readFile/hGetContents` holder 'a semi-closed handle' på filen inntil hele 'innhold'et blir enten konsumert eller utilgjengelig (ut av skopet). Et forsøk på å skrive til en slik fil (f.eks., med `writeFile`) gir vanligvis en feil `isAlreadyInUseError...`

Modifisering av en fil (forts.)

- - Et problem i eksemplene hittil grunner i lat evaluering, dvs.
- - at innholdet av filen ikke har blitt lest/brukt opp.
- - Hvis vi tvinger evaluering av hele innholdet fra filen
- - blir filen lukket og da kan den overskrives.

```
flyttLn fn = do                                -- flytt første linjen til slutt
  innhold <- readFile fn
  let (l1:rest) = lines innhold
      nytt = unlines (rest++[l1])
  if (length innhold > 0) then
    writeFile fn nytt ..... Exception (resource busy...)
  else return ()
```

```
flyttLn fn = do
  innhold <- read' fn                            -- strikt version
  let (l1:rest) = lines innhold
      nytt = unlines (rest++[l1])
  writeFile fn nytt ..... Exception (resource busy...)
```

Modifisering av en fil (forts.) – System.Directory

```
flyttLn3 fn = do
  fh <- openFile fn ReadMode
  content <- hGetContents fh
  let (l1:rest) = lines content
      newContent = unlines (rest++[l1])
  (fn2, fh2) <- openTempFile "." " temp"           System.IO
  hPutStr fh2 newContent
  hClose fh
  hClose fh2
  removeFile fn                                     System.Directory
  renameFile fn2 fn                                 System.Directory
```

```
openTempFile :: FilePath -> String -> IO (FilePath, Handle)

Hva hvis filen fn ikke finnes?
doesFileExist :: FilePath -> IO Bool               System.Directory
  exist <- doesFileExist fn
  if (exist) then ... else return ()
```

Kursor posisjonering – ANSI koder:

"\ESC[..."

putStr "\ESC[2J"

– blank skjermen

putStr "\ESC[0J" / "\ESC[1J" – blank skjermen etter/før kursor

Skjermen:

		x →				
		0	1	2	3	...
y ↓	0	0:0	1:0	2:0	3:0	...
	1	0:1	1:1	2:1	3:1	...
	2	0:2	1:2	2:2	3:2	...
	3	0:3	1:3	2:3	3;3	...
	⋮					

putStr "\ESC[x;yH"

– gå med kursor til posisjonen x:y

putStr "\ESC[3;2H"

– gå med kursor til posisjonen 3:2

ANSI koder:

http://en.wikipedia.org/wiki/ANSI_escape_code

Windows:

<https://docs.microsoft.com/en-us/windows/console/console-virtual-terminal-sequences>

exx = do

```
putStrLn "Hva skal jeg gjøre?"
```

```
c <- getLine
```

```
let com = words c
```

```
if (head com == "av") then return ()
```

```
else if (head com == bs") then do putStr "\ESC[2J" -- blank skjermen  
exx
```

```
else if (head com == bk") then do putStr "\ESC[01J" -- blk ettr.før kursor  
exx
```

```
else if (head com == "ny") then do
```

```
let x = read (head (tail com)) :: Int
```

```
y = read (last com) :: Int
```

```
goto x y
```

```
putStr "X"
```

```
exx
```

```
else do printputStrLn "Ukjent kommando" skriver: "Ukjent kommando"
```

```
exx
```

```
goto x y = putStr ("\ESC[" ++ show y ++ ";" ++ show x ++ "H")
```

```
putStr ("\ESC[5;7H") -- gå med kursor til posisjonen 5:7
```

ANSI koder: "\ESC[..."

(ikke alle OS/term.)

– words:: String -> [String]

Aksjoner og funksjoner

```
les filNavn = do innhold <- readFile' filNavn
               let ord = words innhold
               res = metode ord          – behandler innholdet
               writeFile (filNavn++" 1") res

metode :: String -> String           les :: FilePath -> IO ()
```

!!! En gang uren, alltid uren

? fun :: FilePath -> String – henter filens innhold

a) fun navn = readFile navn readFile :: FilePath -> IO String
fun :: FilePath -> IO String

b) fun navn = do innhold <- readFile navn
 return innhold
fun :: FilePath -> IO String return :: t -> IO t

!!! Aksjoner kan kalle funksjoner (f.eks., inne i 'let v = fun arg'),
men en funksjon som "kaller" en aksjon må selv bli en aksjon
– kode som inneholder en aksjon må selv være en aksjon

Mange morsomme programmeringsoppgaver på forskjellige nivå:

- ▶ løsninger – også i Haskell – kan leveres til testing
- ▶ med IO grensesnitt, dvs. input må leses fra terminal og resultat skrives ut til terminal

```
import System.IO
main = do
    input1 <- getLine
    ...
    inputN <- getLine
    ...
    putStr (show resultat)
```